The Moderator Platform

Overview:

The **Moderator Event Platform** is designed to help moderators efficiently manage and respond to events assigned to their specific regions. The platform ensures that incoming events are quickly claimed by moderators and acknowledged within a defined time window.

Each moderator belongs to a **specific region**, and events generated in that region are displayed only to moderators assigned to that region. This prevents overlap and ensures faster response times.

The system enforces an **event ownership model**, where a moderator must first **claim an event** before acknowledging it. Once an event is claimed, it becomes **locked** for other moderators and must be acknowledged within **15 minutes**.

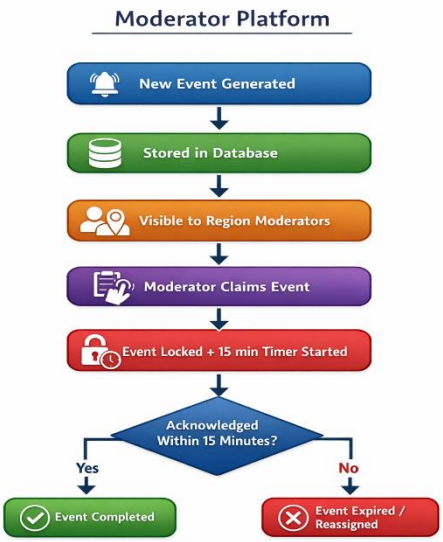
System Architecture:

The platform follows a modern web application architecture consisting of:

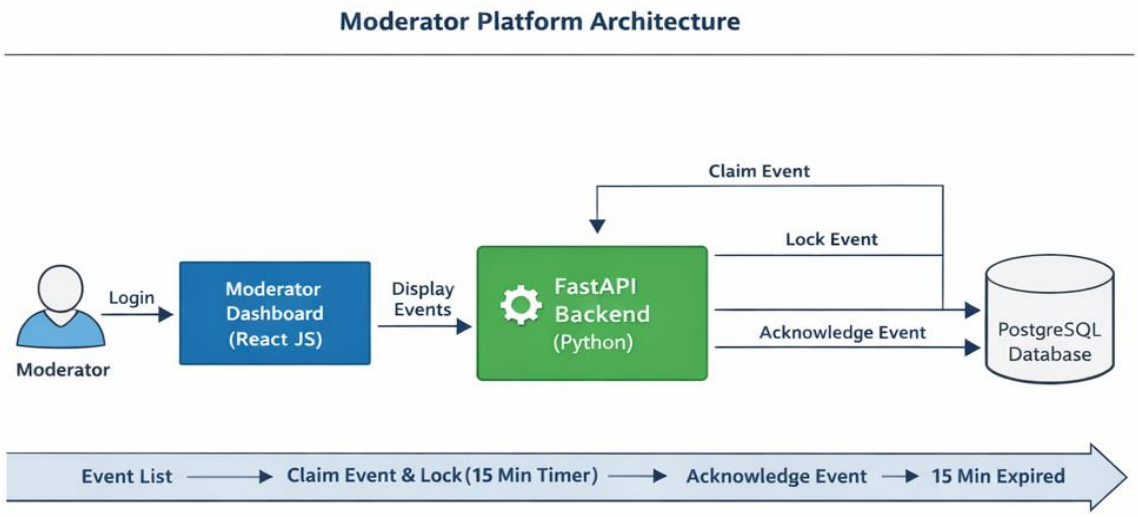
- **Frontend**: React Js - version 19
- **Backend**: Python (Fast API)
- **Database**: PostgreSQL

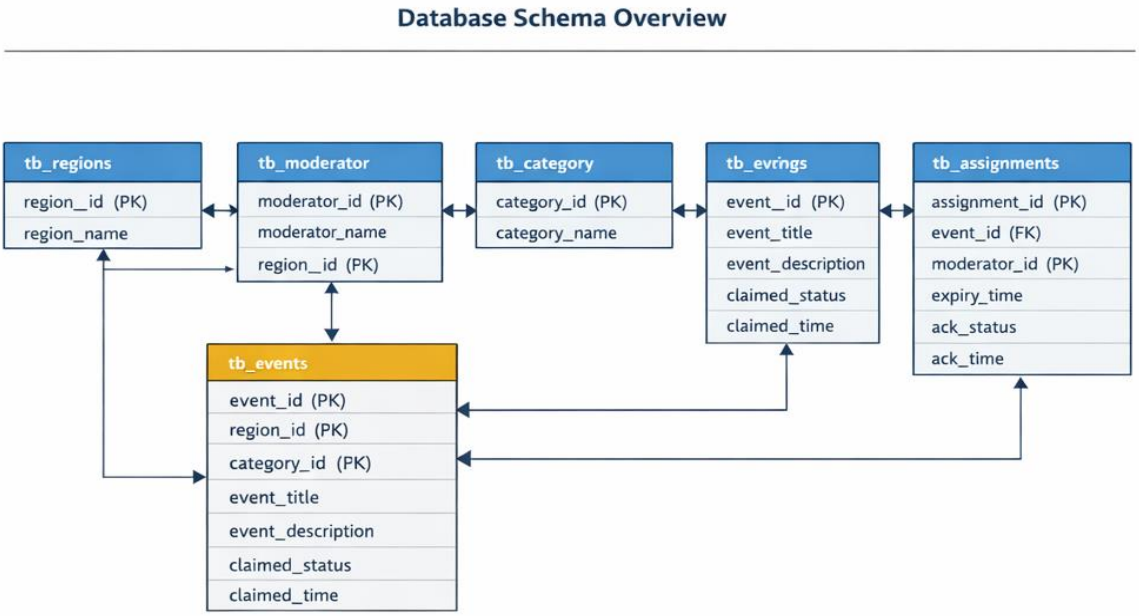
Core System Entities:

- **Moderator** - Represents a user responsible for handling events.
  - **Attributes**: Moderator Name, Region, Login credentials
- **Event** - Represents an event requiring moderator action.
  - **Attributes**: Category, Event title, Event description, Claim event
- **Event Claim** - Tracks ownership of an event
  - **Attributes**: Event ID, Moderator ID, Acknowledge status



Architecture & Database Design:





Workflow:

- **Moderator Login**
    a. Moderators log into the platform using their **name** and **region**.
    b. Each moderator profile contains an associated **region**.
    c. After successful login, the moderator is redirected to the Dashboard.
- **Moderator Dashboard**
    a. Only events belonging to the **moderator's assigned region** are displayed.
    b. Events that are already **claimed by another moderator** are not available for claiming.
- **Event Claiming**
    a. The moderator clicks **Claim Event.**
    b. The system performs the following actions:
        i. Assigns the event to that moderator.
        ii. Locks the event so **other moderators cannot claim it**.
        iii. Expiry Time of 15mins is updated in database
- **Event Acknowledgment**
    a. The moderator must **acknowledge the event within 15 minutes**.
    b. If the moderator acknowledges within the time window, acknowledged status becomes 1
    c. If not acknowledged within 15 minutes, the event is flagged as **missed**

Event Ingestion Approach:

The system uses a scheduler-based ingestion mechanism to continuously load events into the database in controlled batches. This approach ensures:

- System stability under load
- Efficient processing of large datasets
- Controlled database writes

Ingestion Mechanism
A background scheduler runs at dynamic intervals to insert events into the system.

Key Characteristics

- Events are inserted in **batches**
- Scheduler interval **adapts based on system load**
- Load is determined by the number of **unclaimed events**

Parameter	Value
Default interval	15 seconds
Batch size	20 events
Scaling factor	+15 seconds per 1000 unclaimed events

Dynamic Scheduling Logic

The scheduler interval increases as the number of unclaimed events grows.

Formula: interval = 15 + (available_events // 1000) * 15

Design Rationale

- **Adaptive Load Control :** Instead of using a fixed interval, the system dynamically adjusts ingestion frequency based on current backlog preventing database overload and excessive write operations.
- **Batch Processing:** Events are inserted in small batches to reduce transaction size and improve performance.
- **Backpressure Handling:** As unclaimed events increase, ingestion slows down ensuring stability under high volume

API Documentation:

The Moderator Platform exposes REST APIs built using **FastAPI**. These APIs allow moderators to authenticate, retrieve events, claim events, and acknowledge events.
All APIs return responses in **JSON format**.
Base URL : http://127.0.0.1:8000
Authentication: Bearer <JWT_TOKEN> (Except Login API)

1. **Login API**
   a. **Endpoint Path:** /login
   b. **Method**: POST
   c. **Authentication Required:** No (Public API)
   d. **Description**
      The login API authenticates a moderator using the moderator name and region ID. If the credentials are valid, the system generates and returns a JWT token that will be used to access protected APIs.
   e. **Parameters**
      i. **moderator_name**: String
         Description: Name of the moderator (maximum 8 characters)
      ii. **moderator_region_id**: Integer
         Description: Region assigned to the moderator
   f. **Response** : If the login is successful, the API returns a success message along with a JWT authentication token.
   g. **Example**:
      i. Request Body

```
{
    "moderator_name": "user000",
    "moderator_region_id": 1
}
```

ii. Success Response


```
{
    "success": true,
    "message": "Login successful",
    "data": {
        "moderator_id": 1,
        "moderator_name": "user000",
        "region_id": 1,
        "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJtb2
        "token_type": "Bearer"
    }
}
```


2. **Get Events API**
   a. **Endpoint Path:** /events
   b. **Method**: GET
   c. **Authentication Required:** Yes (JWT Token Required)
   d. **Description**
      This API returns the list of events available for the moderator's assigned region. Only events that belong to the moderator's region and are not yet claimed will be displayed.
   e. **Parameters**
      i. **region_id**: Integer (Path parameter)
         Description: Region identifier used to retrieve region-specific events.
   f. **Response:** The API returns a list of events including event ID, title, category, and claim status.
   g. **Example**:
      i. Success Response


```
{
    "success": true,
    "message": "List of Events",
    "data": [
        {
            "event_id": 104,
            "category_name": "Equipment Safety",
            "event_title": "Crane Sensor Malfunction",
            "event_description": "Pallet jack wheel damaged while transporting goods.",
            "claimed_status": 0
        },
        {
            "event_id": 113,
            "category_name": "Equipment Safety",
            "event_title": "Damaged Pallet Jack",
            "event_description": "Forklift brake system reported malfunction during warehouse operation.",
            "claimed_status": 0
        },
        {
            "event_id": 118,
            "category_name": "Transport Safety",
            "event_title": "Improper Trailer Lock",
            "event_description": "Cargo inside truck not secured before transport.",
            "claimed_status": 0
```

3. **Claim Event API**
   a. **Endpoint Path:** /claim
   b. **Method**: POST
   c. **Authentication Required:** Yes (JWT Token Required)
   d. **Description**
      This API allows a moderator to claim an event. Once an event is claimed, it becomes locked and cannot be claimed by any other moderator. At the time of claiming, the system records the claim time and sets an expiry time of 15 minutes.
   e. **Parameters**
      i. **event_id**: Integer
         Description: Unique identifier of the event to be claimed.
      ii. **region_id**: Integer
         Description: Region assigned to the moderator.
      iii. **moderator_id:** Integer
         Description: Unique identifier of the moderator claiming the event.
   f. **Response:** If the claim is successful, the API returns a confirmation message along with the event expiry time.
   g. **Example**:
      i. Request Body:

```
{
    "event_id":104,
    "region_id": 1,
    "moderator_id": 1
}
```

      ii. Success Response:

```
{
    "success": true,
    "message": "Event successfully claimed",
    "data": {
        "event_id": 104,
        "expire_time": "2026-03-18T11:38:35.458171+00:00",
        "event_title": "Crane Sensor Malfunction",
        "event_description": "Pallet jack wheel damaged while transporting goods."
    }
}
```

4. **Acknowledge Event API**
   a. **Endpoint Path:** / acknowledge
   b. **Method**: POST
   c. **Authentication Required:** Yes (JWT Token Required)
   d. **Description**
      This API allows a moderator to acknowledge a claimed event. The acknowledgment must occur within 15 minutes of the event being claimed.
   e. **Parameters**

i. **event_id**: Integer
Description: Unique identifier of the event to be claimed.

ii. **moderator_id:** Integer
Description: Unique identifier of the moderator claiming the event.

f. **Response:** If the acknowledgment is successful, the event status is updated and the system marks the event as completed.

g. **Example**:

i. Request Body:

```
{
    "event_id": 104,
    "moderator_id": 1
}
```

ii. Success Response:

```
{
    "success": true,
    "message": "Event acknowledged.",
    "data": {
        "event_id": 104,
        "ack_time": "2026-03-18T11:26:23.119857+00:00"
    }
}
```

5. **Event Expiry Behavior:**

a. **Endpoint Path:** / expire

b. **Method**: POST

c. **Authentication Required:** Yes (JWT Token Required)

d. **Description**

When a moderator claims an event, the system starts a 15-minute timer. The moderator must acknowledge the event within this time window.

If the event is acknowledged within 15 minutes, the event status changes to acknowledged and the process is completed.

If the event is not acknowledged within 15 minutes, the assignment expires automatically and the event is marked as expired in the system.

e. **Parameters**

i. **event_id**: Integer
Description: Unique identifier of the event to be claimed.

ii. **moderator_id:** Integer
Description: Unique identifier of the moderator claiming the event.

f. **Response:** If the claimed event is expired, the API returns a confirmation message.

Design Decisions:

1. **Clean Architecture** (Router → Service → Repository)
The backend follows a layered architecture:
Router → Service → Repository → Database

2. **Event Expiry Mechanism**
   Each claimed event has a 15-minute expiry window.
   If a moderator does not acknowledge within this time:
   - The event becomes available again.
   - **claimed_status** is reset

   This ensures events do not remain locked indefinitely.
3. **Pagination**
   Pagination was implemented to:
   - Reduced API response size
   - Improved frontend performance
   - Handle large event datasets
4. **Standard API Response Format**
   { "success": true/false,
   "message": "Description",
   "data": {} }

Assumptions:

- Each event belongs to a single region.
- A moderator can claim multiple events.
- Event expiry time is fixed at **15 minutes**.
- Event acknowledgement must occur before the expiry time.
- Only available events are shown in the event listing API.

Extra Credit:

Below points implemented

- Exposing useful metrics like total assigned, currently active, and expired events.
- Implementing background workers or scheduled jobs for expiring stale assignments.
- Writing basic unit or integration tests for critical logic.